Chapitre : Notions préalables à la programmation asynchrone

Synchrone/asynchrone, fonctions comme valeurs, callbacks, génériques, *null safety*, `Iterable`, packages et HTTP

Pr. Abdelfattah Abassi

Département Physique Appliquée – Laboratoire LAVETE
Faculté des Sciences et Techniques de Settat
Université Hassan I

Année universitaire 2026–2027

Objectifs et organisation du chapitre

À la fin de ce chapitre, vous saurez

- distinguer exécution **synchrone** et **asynchrone**, **bloquant** et **non bloquant** ;
- écrire des **fonctions anonymes**, des fonctions **flèche** et des **callbacks** ;
- comprendre les **événements** et l'**abonnement** ;
- lire les types **génériques** (`Future<T>`, `List<T>`) ;
- manipuler `null`, `?`, `??`, `!`, `as`, `is` ;
- distinguer `Iterable` et `Stream` grâce aux générateurs ;
- importer des bibliothèques et des **packages** ;
- comprendre client/serveur, URL et JSON comme texte.

Place dans le cours

À enseigner **avant** le chapitre « Programmation asynchrone, exceptions et API REST ».

Prérequis

Variables, types de base, conditions, boucles, fonctions simples, classes simples, listes et dictionnaires (chapitre « Le langage Dart »).

Méthode

idée → *exemple* → *exercice*.

Pourquoi ces notions avant l'asynchrone ?

Notion préalable	Où elle sert dans le chapitre suivant
Synchrone, bloquant, latence	« Pourquoi l'asynchrone ? », `Future.delayed`
Fonctions anonymes et flèche	`then`, `listen`, `map`, `where`, `catchError`
Callbacks et événements	`listen`, `onError`, `onDone`, boucle d'événements
Paramètres nommés	`onError:`, `headers:`, `body:`, `{required ...}`
Génériques	`Future<T>`, `Stream<T>`, `List<Todo>`
Null safety, `as`, `dynamic`, `num`	`jsonDecode`, `json['id'] as int`, `snapshot.data!`
`Iterable`, `sync*`	`Stream`, `async*`, `yield`, `await for`
Constructeur `factory`, `implements`	`Todo.fromJson`, exceptions personnalisées
Bibliothèques et packages	`dart:async`, `dart:convert`, `package:http`
Client/serveur, URL, texte	API REST, JSON, `statusCode`

Plan du chapitre

1. Synchrone et asynchrone
2. Fonctions comme valeurs
3. Callbacks et événements
4. Types génériques
5. *Null safety* et types dynamiques
6. Collections, `Iterable` et générateurs
7. Classes pour modéliser les données
8. Bibliothèques et packages
9. Réseau et HTTP : les bases
10. Synthèse et auto-évaluation

Plan du chapitre

1. Synchrone et asynchrone
2. Fonctions comme valeurs
3. Callbacks et événements
4. Types génériques
5. *Null safety* et types dynamiques
6. Collections, `Iterable` et générateurs
7. Classes pour modéliser les données
8. Bibliothèques et packages
9. Réseau et HTTP : les bases
10. Synthèse et auto-évaluation

Exécution synchrone : une instruction après l'autre

```
void main() {
  print('1. Je me lève');
  print('2. Je prépare le café');
  print('3. Je bois le café');
}
```

Sortie :

```
1. Je me lève
2. Je prépare le café
3. Je bois le café
```

Synchrone

Chaque instruction **attend** que la précédente soit terminée. L'ordre d'exécution est l'ordre du code.

Analogie : le guichet

Une seule personne est servie à la fois ; les autres **attendent** leur tour.

Toutes les opérations ne durent pas pareil

Opération	**Durée (ordre de grandeur)**
addition, comparaison	quasi instantanée
lecture d'un fichier local	quelques millisecondes
appel réseau (API)	dizaines à centaines de ms, parfois des secondes
mesure d'un capteur (Bluetooth)	100 ms à plusieurs secondes
saisie de l'utilisateur	imprévisible
serveur en panne	potentiellement **infinie**

Pourquoi c'est un problème ?

Un écran de téléphone se rafraîchit **60 fois par seconde** : environ **16 ms** par image.

Attendre 300 ms sans rien faire, c'est perdre une **vingtaine d'images** : l'écran semble figé.

Synchrone = bloquant

```dart
import 'dart:io';

void main() {
  print('Début');
  sleep(Duration(seconds: 2));   // bloque TOUT le programme
  print('Fin');
}
```

Sortie :

```
Début
(2 secondes : rien ne se passe)
Fin
```

sleep

Met le programme en pause : pendant ce temps, **aucune autre instruction** ne peut s'exécuter.

À ne jamais faire dans Flutter

`sleep` sert ici **uniquement** à illustrer le blocage. Dans une application, l'interface serait figée.

Asynchrone = non bloquant

Définition

Opération **asynchrone** : on la **lance**, on **continue** le travail, et on récupère le **résultat plus tard**.

Analogie : la machine à laver

Synchrone : rester debout devant la machine jusqu'à la fin du cycle.
Asynchrone : lancer la machine, faire autre chose, revenir quand elle **sonne**.

Synchrone	**Asynchrone**
on attend sur place	on lance et on continue
programme bloqué	programme réactif
résultat immédiat	résultat plus tard
ordre = ordre du code	l'ordre d'arrivée peut différer
calculs rapides	réseau, capteurs, fichiers

Vocabulaire à maîtriser

Terme	**Signification**
synchrone / asynchrone	on attend le résultat sur place / on le récupère plus tard
bloquant / non bloquant	le programme est figé pendant l'attente / il reste disponible
séquentiel / simultané	une tâche après l'autre / plusieurs tâches en cours en même temps
concurrence	plusieurs tâches **alternent** (un seul fil d'exécution possible)
parallélisme	plusieurs tâches calculent **réellement** en même temps (plusieurs curs)
latence	délai entre la demande et la réponse
callback	fonction fournie pour être appelée **plus tard**
événement	fait qui se produit : clic, message reçu, mesure d'un capteur

Un avant-goût des trois mots-clés

```dart
Future<String> chargerNom() async {
  await Future.delayed(Duration(seconds: 2));
  return 'Sara';
}

void main() async {
  print('Début');
  String nom = await chargerNom();
  print('Bonjour $nom');
}
```

Les trois mots-clés

`Future` : une valeur **plus tard**
`async` : la fonction **peut attendre**
`await` : **attendre** un résultat sans bloquer le reste

Pas de panique

Ces mots-clés sont expliqués **pas à pas** dans le chapitre suivant. Ici, retenez seulement leur **rôle**.

Mesurer le temps : Duration et Stopwatch

```dart
import 'dart:io';

void main() {
  final chrono = Stopwatch();
  chrono.start();
  sleep(Duration(milliseconds: 500));
  chrono.stop();
  print('Durée : ${chrono.elapsedMilliseconds} ms');

  var d = Duration(minutes: 1, seconds: 30);
  print(d.inSeconds);
}
```

Sortie :

```
Durée : 501 ms
90
```

Duration

Représente une **durée** : `seconds`, `milliseconds`, `minutes`, `hours`.

Stopwatch

Chronomètre : `start`, `stop`, `elapsedMilliseconds`.

Exercices : synchrone et asynchrone

1. Écrivez un programme qui affiche trois messages séparés par une pause de 1 s (`sleep`) et mesurez la durée totale avec `Stopwatch`.
2. Classez en *rapide* ou *lent* : trier une liste de 10 nombres, télécharger une image, lire un capteur Bluetooth, calculer `3 * 4`.
3. Expliquez pourquoi un `sleep` de 2 s dans une application mobile **fige l'écran**.
4. Donnez deux exemples de la vie quotidienne **synchrones** et deux **asynchrones**.
5. À 60 images par seconde, combien d'images environ sont perdues pendant une attente de 200 ms ?

Plan du chapitre

1. Synchrone et asynchrone

2. Fonctions comme valeurs

3. Callbacks et événements

4. Types génériques

5. *Null safety* et types dynamiques

6. Collections, `Iterable` et générateurs

7. Classes pour modéliser les données

8. Bibliothèques et packages

9. Réseau et HTTP : les bases

10. Synthèse et auto-évaluation

Une fonction est une valeur

```dart
int doubler(int x) => x * 2;

void main() {
  int Function(int) f = doubler;    // on range la fonction
  print(f(5));                      // 10
  print(doubler(5));                // 10
}
```

Sortie :

```
10
10
```

Idée clé

En Dart, une fonction peut être **rangée dans une variable**, **passée en paramètre** ou **renvoyée**.

Le type d'une fonction

`int Function(int)` : prend un `int`, renvoie un `int`.

`void Function()` : ne prend rien, ne renvoie rien.

`bool Function(String)`

Fonction anonyme et syntaxe flèche

```
// fonction nommée
int carre(int x) {
  return x * x;
}

// fonction anonyme (sans nom)
var carre2 = (int x) {
  return x * x;
};

// syntaxe flèche : une seule expression
var carre3 = (int x) => x * x;

void main() {
  print(carre3(4));        // 16
}
```

La flèche =>

Signifie « **renvoie directement** cette expression ». Pas de `return`, pas d'accolades.

Limite

La flèche accepte **une seule expression**. Pour plusieurs instructions, utilisez des accolades.

Passer une fonction en paramètre

```
void repeter(int n, void Function(int) action) {
  for (int i = 1; i <= n; i++) {
    action(i);                    // on appelle la fonction reçue
  }
}

void main() {
  repeter(3, (i) => print('Tour $i'));
}
```

Sortie :

```
Tour 1
Tour 2
Tour 3
```

Fonction d'ordre supérieur

Une fonction qui **reçoit** une fonction (ou en renvoie une).

Pourquoi ?

On passe un **comportement** : l'appelant décide **quoi faire**, la fonction décide **quand**. C'est le principe de `then((v) => ...)` et `listen((v) => ...)`.

Paramètres positionnels, nommés et requis

```dart
void connecter(String hote, {int port = 80, bool securise = false}) {
  print('$hote:$port (sécurisé : $securise)');
}

void envoyer({required String message}) {
  print('Envoi : $message');
}

void main() {
  connecter('exemple.com');
  connecter('exemple.com', port: 443, securise: true);
  envoyer(message: 'Bonjour');
}
```

Sortie :

```
exemple.com:80 (sécurisé : false)
exemple.com:443 (sécurisé : true)
Envoi : Bonjour
```

Paramètres nommés

Entre `{ }`, appelés avec `nom: valeur`. L'**ordre** n'a pas d'importance.

Deux options

valeur par défaut (`port = 80`) ou `required` (obligatoire).

On les retrouve partout : `onError:`, `headers:`, `body:`.

Exercices : fonctions comme valeurs

1. Écrivez en syntaxe flèche une fonction anonyme qui renvoie le **triple** d'un entier ; rangez-la dans une variable et appelez-la.
2. Écrivez `void appliquer(List<int> l, int Function(int) f)` qui affiche `f` appliquée à chaque élément ; testez avec le carré puis avec le double.
3. Écrivez `void saluer(String nom, {String langue = 'fr'})` qui affiche « Bonjour » ou « Hello ».
4. Donnez le **type fonction** de `(String s) => s.length` et de `() => print('ok')`.
5. Quelle différence entre `f` et `f()` quand `f` est une fonction ?

Plan du chapitre

1. Synchrone et asynchrone

2. Fonctions comme valeurs

3. Callbacks et événements

4. Types génériques

5. *Null safety* et types dynamiques

6. Collections, `Iterable` et générateurs

7. Classes pour modéliser les données

8. Bibliothèques et packages

9. Réseau et HTTP : les bases

10. Synthèse et auto-évaluation

Qu'est-ce qu'un callback ?

```dart
import 'dart:async';

void main() {
  print('A');
  Timer(
    Duration(seconds: 1),
    () => print('C : appelé après 1 s'),
  );
  print('B');
}
```

Sortie :

```
A
B
C : appelé après 1 s
```

Callback (rappel)

Fonction **confiée** à quelqu'un d'autre, qui l'**appellera plus tard**, quand l'événement attendu se produit.

Analogie

« Laissez-moi votre numéro, je vous **rappelle** quand c'est prêt. »

`Timer` déclenche la fonction après le délai ; `main` ne s'est pas arrêté pour autant.

Événements et écouteurs

```dart
class Bouton {
  final List<void Function()> _ecouteurs = [];
  void abonner(void Function() action) {
    _ecouteurs.add(action);
  }
  void cliquer() {
    for (final a in _ecouteurs) {
      a();
    }
  }
}
void main() {
  final b = Bouton();
  b.abonner(() => print('Écouteur 1'));
  b.abonner(() => print('Écouteur 2'));
  b.cliquer();
}
```

Sortie :

```
Écouteur 1
Écouteur 2
```

Vocabulaire

événement : le clic
écouteur (*listener*) : la fonction abonnée
abonnement : `abonner(...)`

Lien avec la suite

Un `Stream` est un flux d'événements dans le temps ; `listen` est l'**abonnement**.

Le piège : les callbacks imbriqués

Avec des callbacks (pseudo-code) :

```
chargerUtilisateur(id, (user) {
  chargerCommandes(user, (cmds) {
    chargerDetails(cmds.first, (d) {
      afficher(d);
    });
  });
});
```

Pyramide d'accolades difficile à lire, erreurs difficiles à traiter.

Avec `await` (chapitre suivant) :

```
final user = await chargerUtilisateur(id);
final cmds = await chargerCommandes(user);
final d    = await chargerDetails(cmds.first);
afficher(d);
```

Même logique, code linéaire.

Conclusion

`Future` et `async`/`await` ont été inventés pour **éviter** cette imbrication : le code asynchrone se lit **comme du code synchrone**.

Exercices : callbacks et événements

1. Avec `Timer`, affichez « Bonjour » après 2 secondes, et un message « Je n'attends pas » immédiatement après l'appel.
2. Écrivez `void telecharger(String nom, void Function(String) quandTermine)` qui appelle `quandTermine` après 1 s (avec `Timer`).
3. Complétez la classe `Bouton` : ajoutez une méthode `desabonner` qui retire un écouteur.
4. Créez une classe `Thermometre` qui prévient ses écouteurs à chaque nouvelle température (`void Function(double)`).
5. Réécrivez le piège des callbacks imbriqués avec trois fonctions de votre choix.